

GitHub Copilot**Lab 2****Instructions:**

- a. Go through the below lab instructions carefully.
 - b. Capture the relevant screenshots of all the steps and its working.
 - c. You are most welcome to use and implement your solutions in the language/tool of your choice.
1. In your existing project/folder, try the below (you can use the "ask" or "plan" or "Agent" mode as appropriate)
 - a. Try to create an entire class with required variables and functions by using Copilot chat window and insert the code into your file.
 - b. Analyse the code created.
 - c. Ask Copilot to do a code review and observe the changes suggested and accept if required
 - d. Try different Models through the model selector to do the above task
 - e. Use the various slash commands: /fix, /explain, /tests etc.,
 - f. Use the Copilot mentions (commands starting with @) in the Github chat window to check on details about vscode, terminal etc.,
 - g. Use comment based code recommendations for adding a new functionality into your code and see if its suggesting multiple options. Click on three dots (...) / more options to open the more suggestions window and see whether multiple suggestions are available. Accept whichever option is suitable.
 2. Once your code is ready, try to give appropriate prompt to Copilot to convert the code to a different language and try to run it. (if it suggests you to add related and recommended additional plugins, check the validity of those plugins and add them.
 3. Try to ask Github copilot chat something un-related to coding and see its response.
 4. Check the current context in the copilot chat window and disable it or add additional context files into the chat as applicable. Run few prompts which might involve multiple added context files.

PYTHON STUDENT CLASS CREATION

Here's a line-by-line explanation:

```
class Student:
```

Defines a new class named `Student`. Everything indented below belongs to this class.

```
def __init__(self, name, age, department):
```

The constructor method — automatically called when a new `Student` object is created. `self` refers to the current object instance.

```
    self.name = name
    self.age = age
    self.department = department
```

Stores the values passed in (`name` , `age` , `department`) as instance attributes on the object, so they can be accessed later.

```
def display_details(self):
```

PYTHON STUDENT CLASS CREATION

Defines a method to display the student's information.

```
    print(f"Name: {self.name}")
    print(f"Age: {self.age}")
    print(f"Department: {self.department}")
```

Prints each attribute using f-strings (formatted string literals that embed variables directly).

```
def calculate_grade(self, marks):
```

Defines a method that takes a `marks` value and determines the letter grade.

```
    if marks >= 90:
        grade = "A"
    elif marks >= 80:
        grade = "B"
    elif marks >= 70:
        grade = "C"
    elif marks >= 60:
        grade = "D"
    else:
```

PYTHON STUDENT CLASS CREATION

```
else:  
    grade = "F"
```

A chain of conditions assigning a letter grade based on the score range.

```
print(f"{self.name}'s grade for {marks} marks: {grade}")  
return grade
```

Prints the result and returns the grade string so the caller can use it.

```
if __name__ == "__main__":
```

This block runs only when the file is executed directly (not when imported as a module).

```
s = Student("Alice", 20, "Computer Science")
```

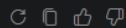
Creates a `Student` object with name "Alice", age 20, department "Computer Science".

```
s.display_details()
```

Calls `display_details()` — prints Alice's name, age, and department.

```
s.calculate_grade(85)
```

Calls `calculate_grade(85)` — evaluates `85 >= 80`, so prints grade "B".



Here's a code review of `student.py` :

Overall: Clean and functional. Minor improvements suggested.

Issues & Suggestions

1. No input validation in `__init__`
`age` and `marks` accept any value — including negatives or strings — without complaint.

```
# Risky:
s = Student("Alice", -5, "CS")
s.calculate_grade(-10) # returns "F" silently, but -10 isn't a real score
```

2. `calculate_grade` should validate `marks` range
 Values like `150` or `-1` are accepted silently. Consider raising a `ValueError`:

```
if not (0 <= marks <= 100):
    raise ValueError(f"marks must be between 0 and 100, got {marks}")
```

PYTHON STUDENT CLASS CREATION

3. Side effect in `calculate_grade` (print + return)
 Methods should generally either print or return, not both — it limits reusability. Prefer returning the grade and letting the caller decide whether to print it.

4. No `__str__` or `__repr__` method
 Without it, `print(s)` gives `<__main__.Student object at 0x...>`. A `__str__` method would make debugging easier:

```
def __str__(self):
    return f"Student({self.name}, {self.age}, {self.department})"
```

5. Grade boundaries are hardcoded
 If grading criteria change, the method needs editing. A dictionary-based approach is more maintainable:

```
GRADE_THRESHOLDS = [(90, "A"), (80, "B"), (70, "C"), (60, "D")]
```

6. `display_details` could return a string instead of printing
 This makes the method testable and reusable in different output contexts (e.g., writing to a file or a web response).

What's Done Well

- Clear, readable structure
- Proper use of `__init__` for attribute initialization

/explain

The code defines a `Student` class that models a student in a school or university system. The class is initialized with three attributes — `name`, `age`, and `department` — which are passed in when creating a new student object and stored using `self` so they persist throughout the object's lifetime.

The `display_details()` method is a simple utility that prints the student's three attributes to the console using f-strings, which allow variable values to be embedded directly inside string literals for clean, readable output.

The `calculate_grade()` method accepts a `marks` parameter and uses a chain of `if/elif/else` conditions to map a numeric score to a letter grade. The thresholds follow a standard grading scale — 90 and above is an "A", 80–89 is a "B", 70–79 is a "C", 60–69 is a "D", and anything below 60 is an "F". The method both prints the result and returns the grade string to the caller.

Finally, the `if __name__ == "__main__":` block serves as the entry point for the script. This guard ensures the code inside only runs when the file is executed directly, not when it is imported as a module into another file. It creates a `Student` object named `Alice`, calls `display_details()` to print her information, and then calls `calculate_grade(85)` which evaluates to a "B" grade.

🔄 📄 👍 📌

student.py:1-31

/fix

Looking at the code, the main issues are:

1. No input validation for `age` and `marks`
2. `calculate_grade` mixes print + return (side effect issue)
3. No `__str__` method for readable object representation

Here's the fixed and improved version:

```
class Student:
    def __init__(self, name, age, department):
        # Validate age
        if not isinstance(age, int) or age <= 0:
            raise ValueError(f"Age must be a positive integer, got {age}")
        self.name = name
        self.age = age
        self.department = department

    def __str__(self):
        return f"Student({self.name}, {self.age}, {self.department})"
```

```
def display_details(self):
    print(f"Name: {self.name}")
    print(f"Age: {self.age}")
    print(f"Department: {self.department}")

def calculate_grade(self, marks):
    # Validate marks range
    if not (0 <= marks <= 100):
        raise ValueError(f"Marks must be between 0 and 100, got {marks}")

    # Grade thresholds
    GRADE_THRESHOLDS = [(90, "A"), (80, "B"), (70, "C"), (60, "D")]
    grade = next((g for threshold, g in GRADE_THRESHOLDS if marks >= threshold), "F")

    return grade

# Example usage
if __name__ == "__main__":
    s = Student("Alice", 20, "Computer Science")
    s.display_details()

    grade = s.calculate_grade(85)
```

```
print(f"{s.name}'s grade: {grade}")

# Test __str__
print(s)

# Test validation
try:
    s.calculate_grade(150)
except ValueError as e:
    print(f"Error: {e}")

try:
    invalid = Student("Bob", -3, "Math")
except ValueError as e:
    print(f"Error: {e}")
```

Key Fixes Summary

Issue	Before	After
Invalid <code>age</code> accepted	No check	Raises <code>ValueError</code> if <code>age <= 0</code>
Invalid <code>marks</code> accepted	No check	Raises <code>ValueError</code> if outside <code>0-100</code>

PYTHON STUDENT CLASS CREATION

Plan:

1. Import `unittest` and `Student` from `day17task.student` (absolute import)
2. Test `__init__` — attributes are set correctly
3. Test `display_details` — captures stdout and verifies output
4. Test `calculate_grade` — all grade boundaries (A/B/C/D/F), edge cases (90, 80, 70, 60, 59, 0, 100)
5. Test return value of `calculate_grade`

```
import unittest
from io import StringIO
from unittest.mock import patch
from day17task.student import Student

class TestStudentInit(unittest.TestCase):
    def test_name_is_set(self):
        s = Student("Alice", 20, "Computer Science")
        self.assertEqual(s.name, "Alice")

    def test_age_is_set(self):
        s = Student("Alice", 20, "Computer Science")
        self.assertEqual(s.age, 20)
```

PYTHON STUDENT CLASS CREATION

```
    def test_department_is_set(self):
        s = Student("Alice", 20, "Computer Science")
        self.assertEqual(s.department, "Computer Science")

class TestDisplayDetails(unittest.TestCase):
    def setUp(self):
        self.student = Student("Alice", 20, "Computer Science")

    def test_display_shows_name(self):
        with patch("sys.stdout", new=StringIO()) as fake_out:
            self.student.display_details()
            self.assertIn("Alice", fake_out.getvalue())

    def test_display_shows_age(self):
        with patch("sys.stdout", new=StringIO()) as fake_out:
            self.student.display_details()
            self.assertIn("20", fake_out.getvalue())

    def test_display_shows_department(self):
        with patch("sys.stdout", new=StringIO()) as fake_out:
            self.student.display_details()
            self.assertIn("Computer Science", fake_out.getvalue())
```

PYTHON STUDENT CLASS CREATION

```

def test_display_full_output(self):
    with patch("sys.stdout", new=StringIO()) as fake_out:
        self.student.display_details()
        output = fake_out.getvalue()
        self.assertIn("Name: Alice", output)
        self.assertIn("Age: 20", output)
        self.assertIn("Department: Computer Science", output)

class TestCalculateGrade(unittest.TestCase):
    def setUp(self):
        self.student = Student("Bob", 22, "Mathematics")

    # --- Return value tests ---
    def test_grade_A_at_100(self):
        self.assertEqual(self.student.calculate_grade(100), "A")

    def test_grade_A_at_90(self):
        self.assertEqual(self.student.calculate_grade(90), "A")

    def test_grade_A_at_95(self):
        self.assertEqual(self.student.calculate_grade(95), "A")

```

PYTHON STUDENT CLASS CREATION

```

def test_grade_B_at_80(self):
    self.assertEqual(self.student.calculate_grade(80), "B")

    def test_grade_B_at_85(self):
        self.assertEqual(self.student.calculate_grade(85), "B")

    def test_grade_B_at_89(self):
        self.assertEqual(self.student.calculate_grade(89), "B")

    def test_grade_C_at_70(self):
        self.assertEqual(self.student.calculate_grade(70), "C")

    def test_grade_C_at_75(self):
        self.assertEqual(self.student.calculate_grade(75), "C")

    def test_grade_C_at_79(self):
        self.assertEqual(self.student.calculate_grade(79), "C")

    def test_grade_D_at_60(self):
        self.assertEqual(self.student.calculate_grade(60), "D")

    def test_grade_D_at_65(self):
        self.assertEqual(self.student.calculate_grade(65), "D")

```


PYTHON STUDENT CLASS CREATION

```

def test_grade_D_at_69(self):
    self.assertEqual(self.student.calculate_grade(69), "D")

def test_grade_F_at_59(self):
    self.assertEqual(self.student.calculate_grade(59), "F")

def test_grade_F_at_0(self):
    self.assertEqual(self.student.calculate_grade(0), "F")

def test_grade_F_at_30(self):
    self.assertEqual(self.student.calculate_grade(30), "F")

# --- Print output tests ---
def test_calculate_grade_prints_name(self):
    with patch("sys.stdout", new=StringIO()) as fake_out:
        self.student.calculate_grade(85)
        self.assertIn("Bob", fake_out.getvalue())

def test_calculate_grade_prints_marks(self):
    with patch("sys.stdout", new=StringIO()) as fake_out:
        self.student.calculate_grade(85)
        self.assertIn("85", fake_out.getvalue())

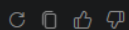
```

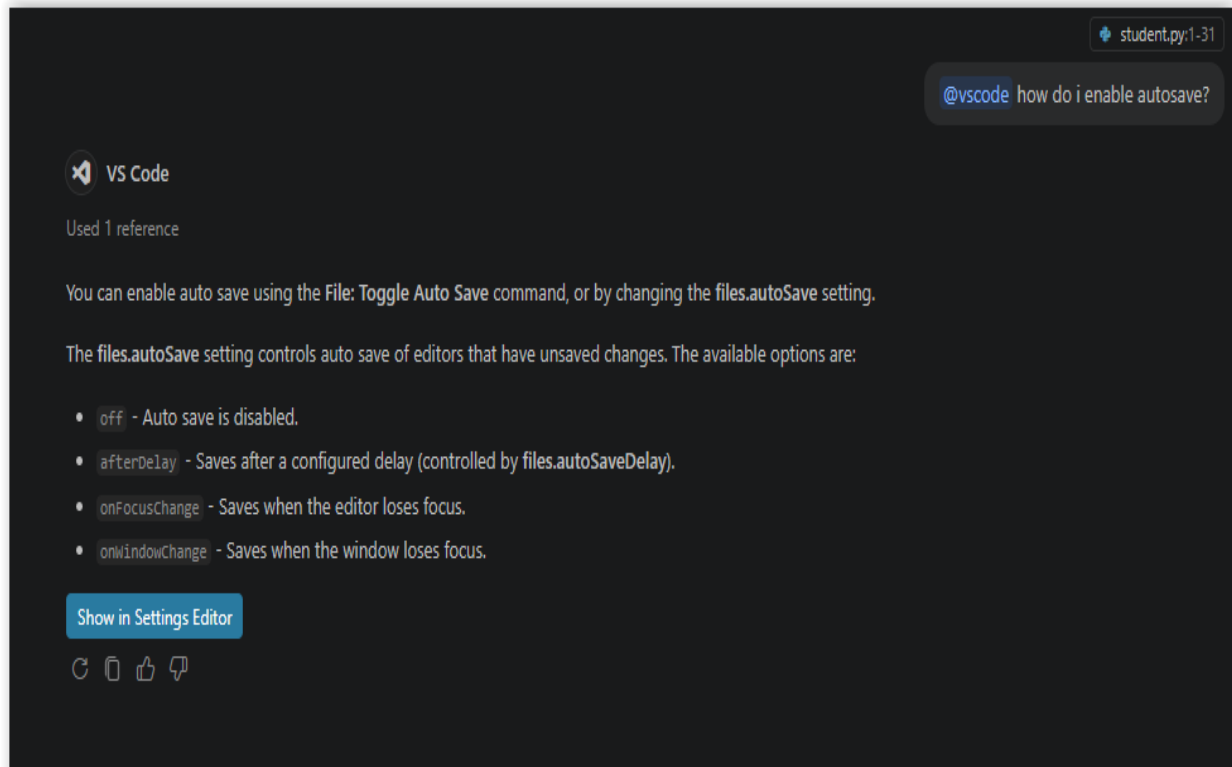
```

def test_calculate_grade_prints_grade(self):
    with patch("sys.stdout", new=StringIO()) as fake_out:
        self.student.calculate_grade(85)
        self.assertIn("B", fake_out.getvalue())

if __name__ == "__main__":
    unittest.main()

```





student.py:1-31

create a function to calculate factorial

Updated student.py +11 -0

Added `calculate_factorial(n)` to `student.py`. It:

- Raises `ValueError` for negative inputs
- Returns `1` for `0` and `1` (base cases)
- Iteratively multiplies from `2` to `n` for all other values

convert this python code to java

Reviewed student.py and created Student.java

Created `Student.java`. Here's a summary of the key Python → Java conversions:

Python	Java
<code>class Student:</code>	<code>public class Student</code>
<code>def __init__(self, ...)</code>	<code>public Student(...) (constructor)</code>
<code>self.name</code>	<code>this.name</code> (with <code>private</code> fields)
<code>def display_details(self)</code>	<code>public void displayDetails()</code>
<code>def calculate_grade(self, marks)</code>	<code>public String calculateGrade(int marks)</code>
<code>def calculate_factorial(n)</code> (module-level)	<code>public static long calculateFactorial(int n)</code>
<code>raise ValueError(...)</code>	throw new <code>IllegalArgumentException(...)</code>
<code>if __name__ == "__main__":</code>	<code>public static void main(String[] args)</code>
f-strings	String concatenation with <code>+</code>

tell me an interesting fact about football

Sorry, I can only help with programming and coding-related questions. For football facts, try a general search engine or a trivia website.

can you improve both files together?

Reviewed student.py and updated prime.py +36 -27

Both files improved. Here's a summary of changes:

prime.py

Before	After
Bare script with a <code>flag</code> variable	Wrapped in <code>is_prime(num)</code> function
Checks all numbers up to <code>n</code> — $O(n)$	Checks up to <code>√n</code> using <code>math.isqrt</code> — $O(\sqrt{n})$
No input error handling	<code>try/except ValueError</code> for invalid input
No <code>__main__</code> guard	Added <code>if __name__ == "__main__"</code>

> Todos (2/2)

> 2 files changed +36 -27

Keep Undo

student.py

Before	After
No validation on <code>age</code>	Raises <code>ValueError</code> if <code>age <= 0</code>
No validation on <code>marks</code>	Raises <code>ValueError</code> if <code>marks</code> outside <code>0-100</code>
<code>if/elif</code> chain for grades	Cleaner threshold-based lookup with <code>next()</code>
<code>calculate_grade</code> prints internally	Returns grade only — caller controls output
No <code>__str__</code> method	Added <code>__str__</code> for readable object representation
Example didn't print grade result	Now prints grade after the call

🔄 📄 👍 🗣️

> Todos (2/2)

> 2 files changed +36 -27

Keep Undo